



Core Architecture, AWS Provisioning & Reuse

Building and scaling highly reliable infrastructure on AWS with Terraform.



Your Name Here

Instructor, Pluralsight



Pluralsight

About Me:

- Instructor
- Bullet 2
- Bullet 3
- Bullet 4
- Bullet 5



Skills coverage across your entire org



SOFTWARE DEVELOPMENT

140+ SKILL ASSESSMENTS | 1600+ COURSES | 450+ LABS | SANDBOXES

- Frontend web development
 - Backend web development
 - Mobile development
 - Software testing
 - Software architecture
 - Version control
- Programming languages:**
- Java
 - Python
 - JavaScript
 - C#, C++
 - Ruby
 - Swift
 - Kotlin
 - TypeScript
 - PHP
 - Go

PRODUCT + BUSINESS MANAGEMENT

35+ SKILL ASSESSMENTS | 900+ COURSES

- Business professionals
- Business analysis
- Communication skills
- Leadership skills
- Portfolio, program, and project management
- Productivity apps and client OS
- User experience design
- Product
- Agile
- Interactive experiences

CLOUD

90+ SKILL ASSESSMENTS | 1100+ COURSES | 2900+ LABS | SANDBOXES

- Amazon Web Services
- Microsoft Azure
- Google Cloud Platform
- Multi-cloud
- Certifications
- Cloud literacy
- Cloud transformation
- Cloud management and monitoring
- Cloud security
- Serverless workloads
- Big data/ML/AI in the cloud
- Cloud identity access management
- Cloud cost control and optimization
- DevOps in the cloud
- Cloud compute and containerization
- Cloud network management and configuration

GENERATIVE AI

20 SKILL ASSESSMENTS | 450+ COURSES | 50+ LABS | SANDBOXES

- Generative AI
- Languages and Libraries
- Chat Bots
- Speech Recognition
- Image Recognition
- Sentence Classification

SECURITY

50+ SKILL ASSESSMENTS | 600+ COURSES | 350+ LABS | SANDBOXES

- Certifications
- Security literacy and fundamentals
- Security architecture and engineering
- Security development
- Security operations
- Security testing
- Governance, compliance, and risk
- Advanced persistent training

IT OPS

50+ SKILL ASSESSMENTS | 1300+ COURSES | 1600+ LABS | SANDBOXES

- IT fundamentals and practices
- Identity management
- Configuration management
- Containers
- DevOps
- Endpoint management
- IT automation
- Management and monitoring
- Messaging and collaboration
- Mobile device management
- Network infrastructure
- Operating systems and stacks
- Server infrastructure
- Storage
- Virtualization
- Container orchestration
- Desktop administration
- Salesforce

DATA

90+ SKILL ASSESSMENTS | 700+ COURSES | 450+ LABS | SANDBOXES

- Data analytics
- Business analytics
- Data engineering
- Data management
- Data science
- Data literacy
- Machine learning
- Managerial skills

*The numbers displayed include all content accessible on our platform and are subject to change.
**Labs and skill assessments may be tagged to multiple domains. This may lead to a display number that is higher than the total sum of labs or skill assessments.

Let's get started!



Proprietary and confidential



Setting Up Your Learning Environment



CLI

Primary interface for all Terraform operations



Text Editor

VS Code or Sublime with HCL syntax support



AWS Fundamentals

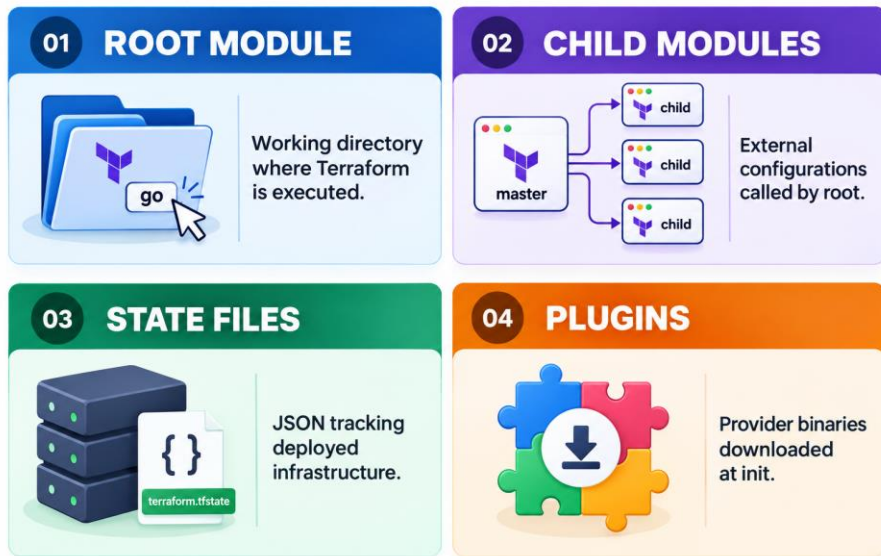
EC2, IAM, VPC -know the basics before you code



Goal: a standardised, functional environment before writing a single line of code.



Setting Up Your Learning Environment



How It Fits Together

The root module orchestrates everything -it calls child modules, writes state, and downloads plugins on `init`.

Root Module

Your working directory

Child Modules

Reusable sub-configurations

State + Plugins

Track reality & talk to APIs



Terraform Project Files

main.tf

Primary entry point -all resource definitions live here

variables.tf

Input variable declarations

outputs.tf

Values returned after a successful apply

 **Convention over configuration -logical separation keeps large codebases readable and maintainable.**

HCL -HashiCorp Configuration Language

Blocks

Containers for content

```
resource "aws_instance"  
  "web" { }
```

Arguments

Assign a value to a name

```
ami = "ami-12345"
```

Expressions

Represent values directly or via computation





YAML in Terraform

Use `yamldecode()` to parse external YAML files into native HCL structures.



Integration

Feed complex app configs from
`.yaml` files into resources

Typical Use Case

Reading microservice settings
without duplicating them in HCL



Terraform Types

Primitive

`string`

`number`

`bool`

Collection & Structural

`list(...)`

Ordered sequence of
values

`map(...)`

Key-value pairs

`object(...)`

Named attributes with
defined types

`tuple(...)`

Typed positional
sequence



Resource Blocks

The core building block -every piece of infrastructure is a resource.

01

Declare

```
resource "aws_instance"  
  "app_server" { ... }
```

02

Configure

Set arguments: `ami,`
`instance_type, tags`

03

Lifecycle

Terraform creates, updates, or destroys to match config





Core Commands

1

`init`

Prepare working directory

2

`plan`

Preview changes

3

`apply`

Execute changes

4

`destroy`

Tear down infrastructure



terraform init



- **First step -run in any new or cloned repo**
- **Downloads provider plugins (e.g. AWS)**
- **Configures backend for state storage**
- **Safe to run multiple times - idempotent**



Extending Your Project

Input Variables

Parameterise configs across environments

Local Values

Avoid repeating complex expressions

Data Sources

Query existing infrastructure dynamically

Move beyond hardcoded scripts -build configs reusable across Dev, QA, and Prod.



Input Variables

Declaration `-variables.tf`

```
variable "region" {  
  type      = string  
  default    = "us-east-1"  
  description = "AWS deployment region"  
}
```

Usage in code

```
provider "aws" {  
  region = var.region  
}
```

type

Enforces accepted value types

default

Fallback when no value is supplied

description

Documents intent for teammates



Assigning Variable Values

01

Environment Variables

`TF_VAR_region=us-east-1` -lowest
precedence

02

Variable Files

`terraform.tfvars` or `*.auto.tfvars`

03

CLI Flags

`-var="region=us-east-1"` -highest
precedence



CLI flags always win. Files beat environment variables.



Local Values

Definition

```
locals {  
  common_tags = {  
    env      = var.env  
    project = "web"  
  }  
}
```

Usage

```
tags = local.common_tags
```

DRY Code

Define once, reference everywhere

Clarity

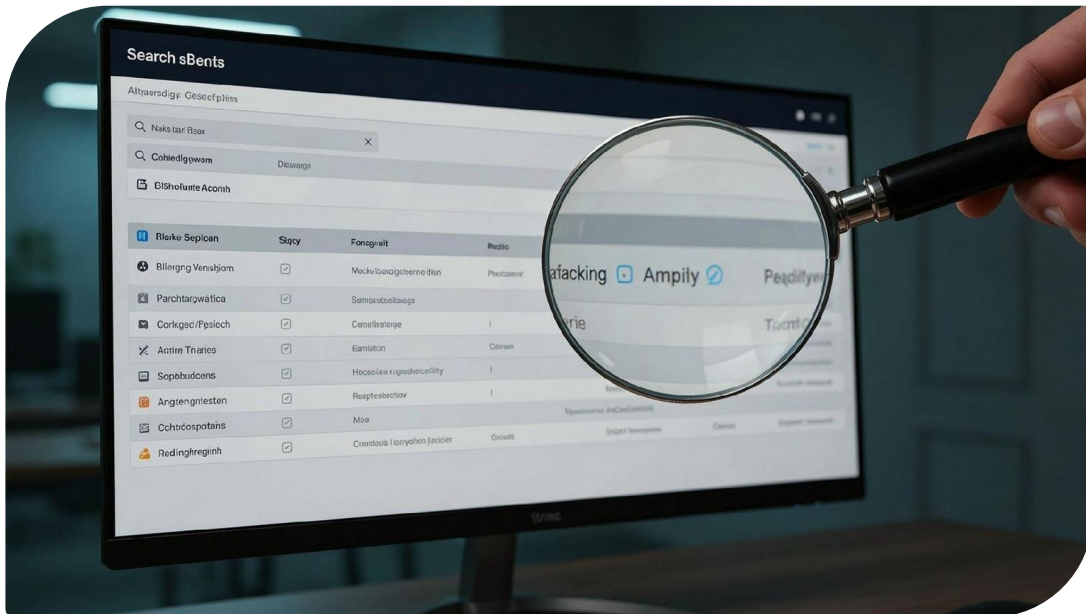
Name complex expressions for readability

No User Input

Unlike variables -computed internally



Data Sources



Query infrastructure not managed by this configuration -e.g. the latest Amazon Linux AMI.

```
data "aws_ami" "latest" {  
    most_recent = true  
    owners      = ["amazon"]  
}
```

Reference it as

`data.aws_ami.latest.id` -always fresh, never hardcoded.



Variable Validation

```
variable "instance_type" {  
  type = string  
  
  validation {  
    condition = contains(  
      ["t3.micro", "t3.small"],  
      var.instance_type  
    )  
    error_message = "Must be t3.micro or  
t3.small."  
  }  
}
```

Condition

Expression that must
evaluate to `true`

Error Message

Custom, human-
readable feedback on
failure

Fail Fast

Catches bad data *before* a plan runs



Provisioners: local-exec



Runs a command on the machine executing Terraform after a resource is created.

Use Case

Trigger an Ansible playbook or a local setup script

Warning

Adds complexity -treat as a last resort



Provisioners: remote-exec

Execution Target

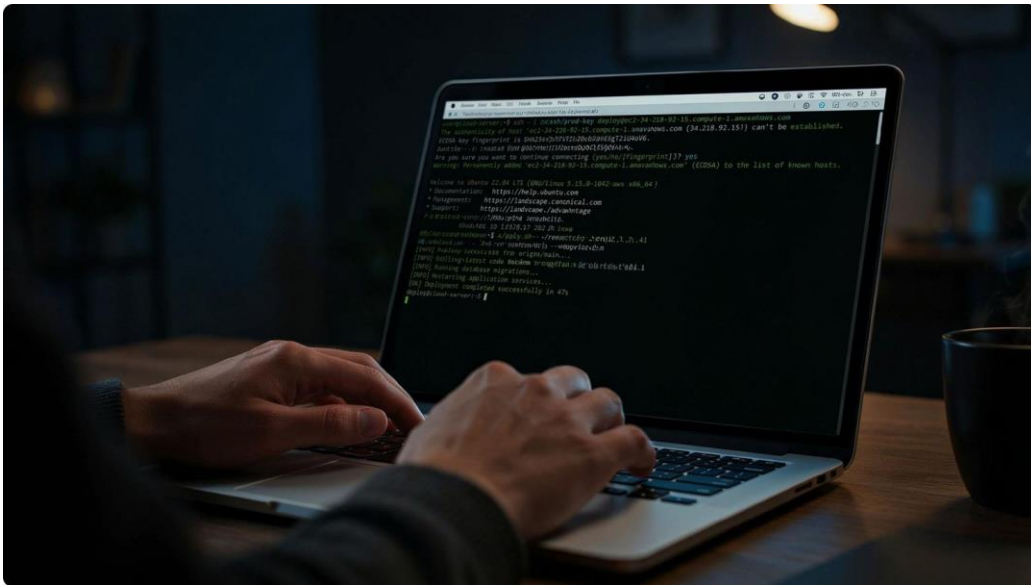
Commands run *on* the remote resource (SSH or WinRM)

Use Case

Bootstrap a newly provisioned EC2 instance

Better Alternative

Prefer cloud-init / user_data for AWS workloads





Provisioners: null_resource

What It Is

A resource that manages nothing natively -a pure provisioner container

Use Case

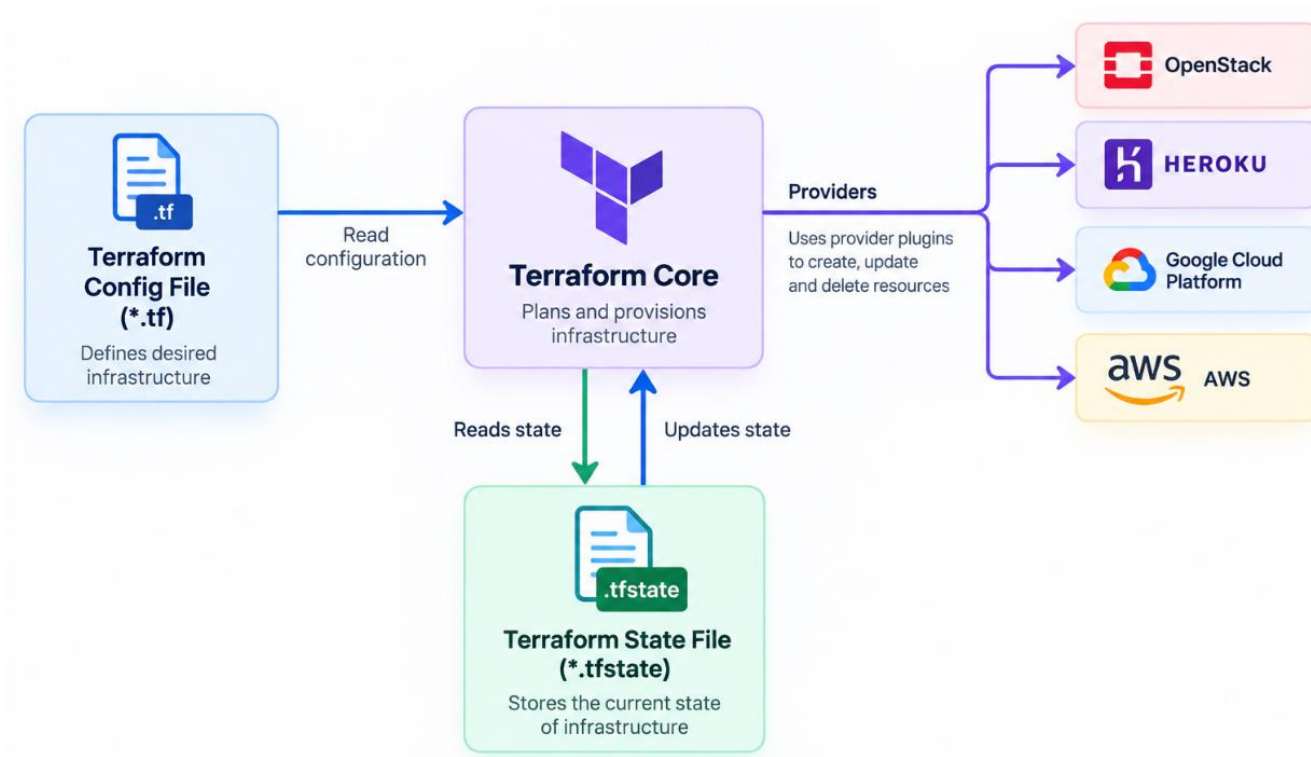
Group `local-exec` or `remote-exec` provisioners without tying them to a real resource

Triggers

Re-run when referenced variables or resource attributes change



How Terraform Works





Terraform State



`terraform.tfstate`

JSON mapping of config to real-world resources



Source of Truth

Terraform reads this to know what it manages



Never Edit Manually

Corruption leads to drift and unrecoverable state



Inspecting the State File

Key Commands

```
terraform state  
list
```

Show all managed
resources

```
terraform state  
show <resource>
```

Inspect attributes of a
specific resource

JSON Format

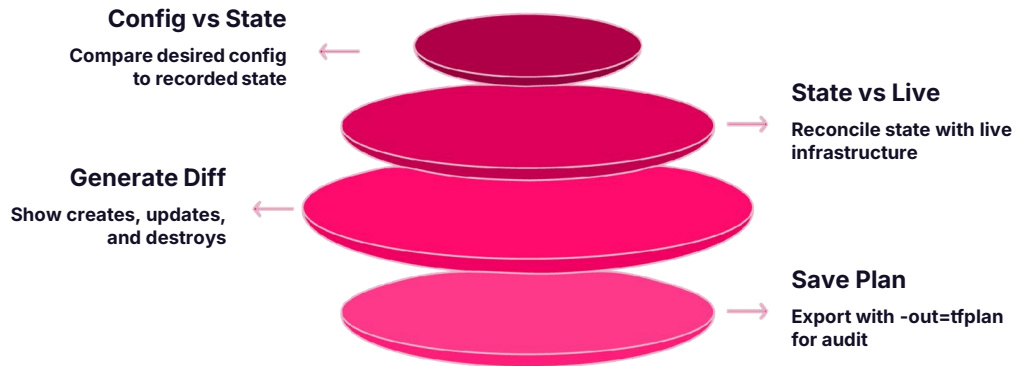
Plain-text by default -
encrypt your remote
backend

Read-Only Workflow

Always use CLI
commands -never open
the file directly



Computing Plans



`terraform plan` -the dry run

- Compares config → state → real infrastructure
- Shows exactly what will change before anything happens
- Save the plan file with `-out=tfplan` for auditability



Reading Plan Output

+ Create

Resource will be provisioned for the first time



~ Update

Modified in-place -no replacement needed

- Destroy

Resource will be permanently removed



-/+ Replace

Must be destroyed and recreated to apply the change



Executing Plans

`terraform apply`

01

Generate Plan

Re-computes the diff

02

Confirm

Type `yes` (or pass `-auto-approve`)

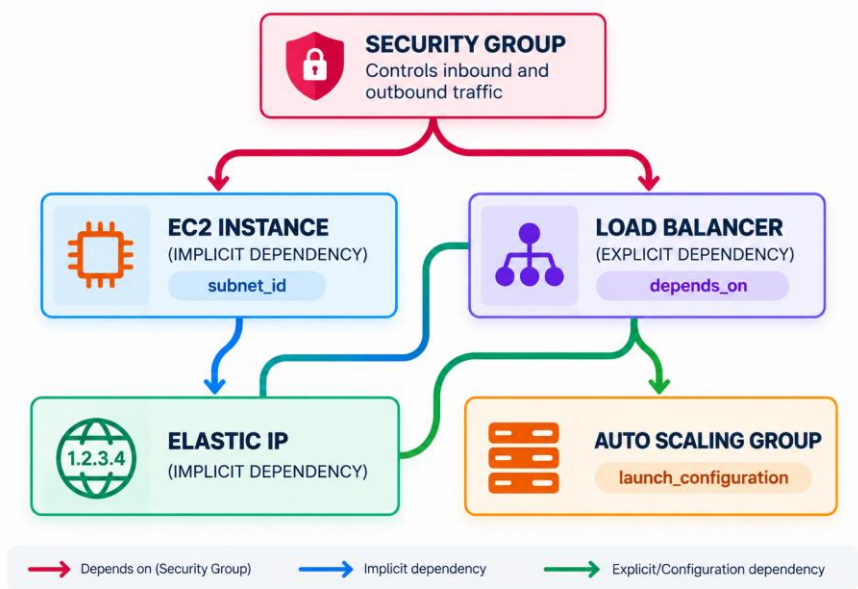
03

Update State

State file written on successful completion



The Resource Graph



Implicit Dependencies

Terraform detects references automatically - SG before EC2

Explicit Dependencies

Use `depends_on` to force ordering when references aren't obvious

Parallelism

Independent resources are created concurrently



Provider Overview



The Interface

Bridges Terraform Core to external APIs -AWS, Azure, GitHub, and hundreds more



Initialisation

Downloaded automatically during `terraform init`



Configuration

Requires auth credentials and regional settings



The AWS Provider

Configuration Block

```
provider "aws" {  
  region = "us-east-1"  
}
```

Authentication Options

- Environment variables
- Shared credentials file
- IAM Role (recommended)

Never Do This

Hardcoding
`access_key` and
`secret_key` in config
files

Best Practice

**Use IAM Roles or
environment variables**
-keep secrets out of
version control



AWS Provider Constraints

1

Version Pinning

```
required_providers {  
  aws = {  
    source = "hashicorp/aws"  
    version = "~> 4.0"  
  }  
}
```

2

Multi-Region Aliases

```
provider "aws" {  
  alias = "west"  
  region = "us-west-2"  
}
```

- ❏ Pin provider versions to guarantee consistent behaviour across all team members.



Templates in Terraform

Generate dynamic config files -JSON, bash scripts, YAML -at plan time.

`templatefile()`

Reads a file and renders it with supplied variables

Typical Use Case

Injecting `var.app_port` into an EC2 `user_data` bootstrap script



Template Rendering

Syntax

```
user_data = templatefile(  
    "${path.module}/script.sh",  
    { port = var.app_port }  
)
```

01

Read File

Terraform reads `script.sh` at plan time

02

Inject Variables

Replaces template placeholders with supplied map values

03

Return String

Rendered output is passed directly to the resource attribute



Configuring EC2 Instances

Resource

`aws_instance`

Required

`ami + instance_type`

Networking

`subnet_id + vpc_security_group_ids`



EC2 Arguments vs. Attributes

Arguments -supplied by you

`ami`

`instance_type`

`key_name`

`tags`

`public_ip`

`arn`

`id`

Attributes -known only after creation

Reference attributes in outputs or other resources:

`aws_instance.web.public_ip`



AWS Authentication Methods

1

Environment Variables

`AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY`

2

Shared Credentials File

`~/.aws/credentials` -standard AWS CLI config

3

IAM Roles

Most secure -use when running Terraform inside AWS
(EC2, CodeBuild)



Reuse Patterns in Terraform

The Problem

Copy-pasting directories for Dev, Staging, and Prod -fragile and error-prone

The Solution

Workspaces for state isolation · Modules for config reuse

1. Modules

Reuse and encapsulate infrastructure code.



```
module "vpc" {  
  source = "../modules/vpc"  
  cidr   = "10.0.0.0/16"  
}
```

- ✓ Use modules to reuse across projects and environments.

2. Variables

Parameterize configurations for reusability.



```
variable "instance_type" {  
  type    = string  
  default = "t3.micro"  
}
```

- ✓ Use variables to make configurations flexible and reusable.

3. Locals

Define reusable values within a configuration.



```
locals {  
  common_tags = {  
    Environment = "dev"  
    Project     = "webapp"  
  }  
}
```

- ✓ Use locals to avoid repetition and keep values consistent.

4. Data Sources

Reuse existing infrastructure data.



```
data "aws_ami" "ubuntu" {  
  most_recent = true  
  owners      = ["099720109477"]  
  filter {  
    name   = "name"  
    values = ["ubuntu-*-lts"]  
  }  
}
```

- ✓ Use data sources to reference existing resources and avoid duplication.



Key Takeaway: Modules, variables, locals and data sources help you write DRY, maintainable and reusable Terraform code.



DRY Principle -Don't Repeat Yourself. Write once, deploy everywhere.



Terraform Workspaces

What Workspaces Provide

Multiple State Files

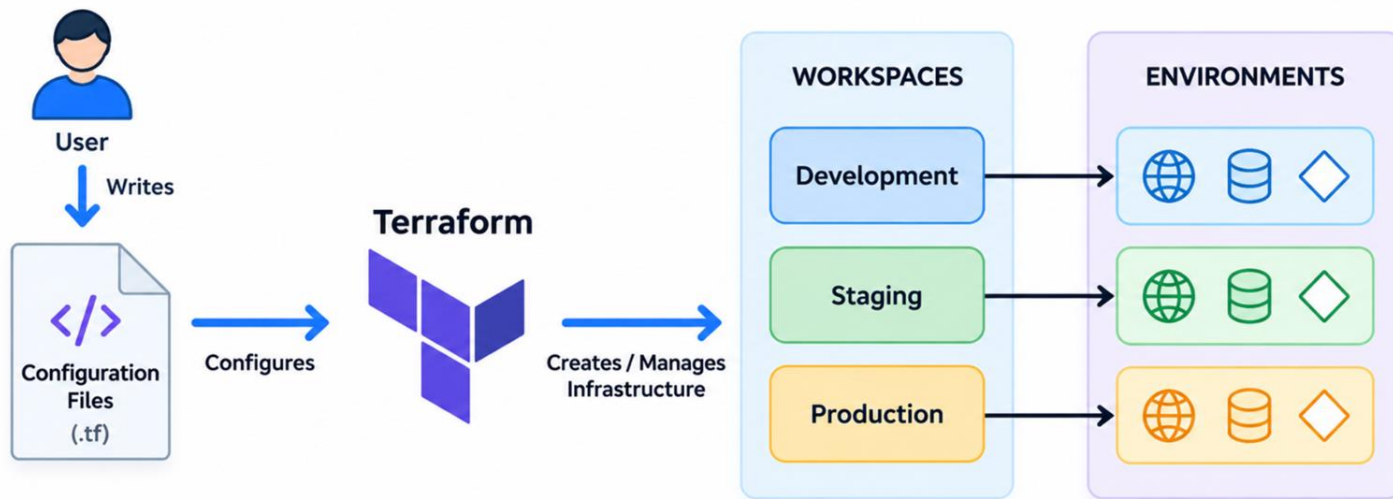
One config directory →
many isolated states

Isolation

Changes in `dev` never touch
`prod`

Ideal Use Case

Testing vs. staging with identical infrastructure shape





Workspace Execution

Create

```
terraform workspace new  
dev
```

Switch

```
terraform workspace  
select prod
```

Conditional Logic

```
instance_type = (  
    terraform.workspace ==  
    "prod"  
    ? "t3.large"  
    : "t3.micro"  
)
```




Outputs

Syntax

```
output "public_ip" {  
    value = aws_instance.web.public_ip  
}
```



Surface Data

Print values after `apply` completes



Module Data Flow

Pass values from child modules back to the root



Introduction to Modules

Root Module

Your main working directory
-the entry point

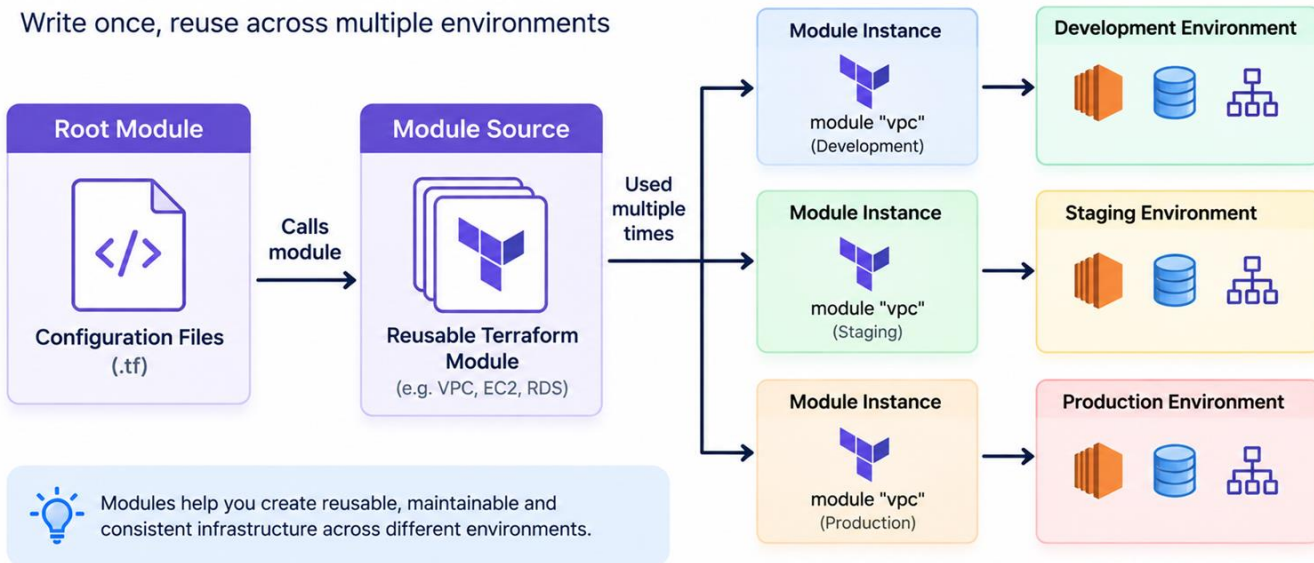
Child Module

Any config called by the root
via a `module` block

Benefits

Encapsulation · Reusability
· Team standardisation

Write once, reuse across multiple environments





Local Module Sources

Reference with a relative path

```
module "vpc" {  
  source = "../modules/vpc"  
  
  cidr_block = var.vpc_cidr  
}
```

Organisation

**Keep project-specific
modules inside the
same repository**

Typical Structure

```
./modules/vpc ·  
./modules/ec2 ·  
./modules/rds
```



External Module Sources



Terraform Registry

```
source = "terraform-aws-modules/vpc/aws"
```



GitHub

```
source = "github.com/hashicorp/example"
```

Leverage community-tested patterns instead of reinventing the wheel for common AWS infrastructure.



Nested Directory Modules

The Double-Slash Convention

```
module "vpc" {  
  source =  
  "github.com/hashicorp/example//modules/vpc"  
}
```

// separates the repository root from the module subdirectory.

Why It Matters

Large repos contain many modules -target exactly the one you need

Works With

GitHub, GitLab, Bitbucket, and generic Git sources



Module Versioning

Registry -version argument

```
module "vpc" {  
  source = "terraform-aws-modules/vpc/aws"  
  version = "~> 3.0"  
}
```

Git -ref tag

```
source = "github.com/org/repo?ref=v1.2.0"
```

Stability

Prevents upstream changes from breaking production

Auditability

Makes it clear exactly which version is deployed

Best Practice

Always pin module versions in production.

The Terraform Registry



The Hub

`registry.terraform.io` -the central home for providers and modules



Verified Modules

Maintained by HashiCorp or official partners -e.g. the AWS VPC module



Discovery

Find, document, and standardise on community-tested infrastructure patterns



Questions?



Thank you

